BREAK THROUGH TECH AI STUDIO · BANKING 1B · NOVEMBER TASK

Baseline Models for Fraud Detection

What Jennifer, Jean and Eric built, why each step exists, and what we learned

Authors: Jennifer Perez-Juarez, Jean Cho, Kyaw Thi (Eric) Ha · November 2026

Notebook: `notebooks/BTT_Mastercard_Baseline_Models_Combined.ipynb`

[Jennifer] Hi team. Our November task was to **build baseline models, compare them fairly, and find out which of our engineered features actually help**. The three of us worked in separate notebooks, which are now combined into one. This document explains every step in plain language, so you can follow it even if you have never trained one of these models. Each part is tagged with who did it: **[Jennifer]**, **[Jean]** or **[Eric]**.

The one-minute summary

- **Boosted trees win clearly.** LightGBM reaches AP **0.632** and XGBoost **0.613**, against **0.428** for logistic regression. Random guessing scores 0.031.
- **LightGBM stays about 0.02 AP ahead of XGBoost**, even after we fixed XGBoost's settings (that fix alone moved it from 0.569 to 0.613). Blending the two did not help.
- **Neural networks land in the middle.** Eric's best network (with learned embeddings for ID-like columns) scores **0.556** over 3 seeds: much better than logistic regression, about 0.075 below LightGBM.
- **Two very different logistic regressions scored exactly the same** (0.4276). Jean's and Jennifer's pipelines agree, so about 0.43 looks like the limit for a linear model on these features.
- **Our engineered feature families gave mixed results.** Removing target encoding or behaviour features helped on one seed and hurt on another. We are not dropping anything yet.
- **About a third of fraud is missed** at a 5% review budget, mostly Product W and transactions without identity data. The next gain probably needs new information, not a new algorithm.
- **Recommendation:** keep LightGBM as the reference model, keep XGBoost (integer-coded categories) as the close second, and keep logistic regression and the embedding network as documented baselines.

All numbers are on the **validation** period (59,054 transactions). The holdout was never loaded.

Who did what

Person	Work	Sections
Jennifer	Research plan and hypotheses, data loading and checks, the shared `evaluate()` function, LightGBM with feature ablations, the team logistic regression, the full XGBoost deep-dive, follow-up experiments, decision log, and merging the three notebooks	0–14
Jean	A second logistic-regression pipeline: her own preprocessing layer, a regularization grid across three feature sets, and a fixed review threshold	B2.0–B2.5
Eric	Small neural networks: size, fraud weighting, embeddings, learning curves, seed stability, blending, calibration, segment and overlap analysis	D0–D19

1 What questions we were answering

Our feature-engineering notebook produced hundreds of columns: target encodings, account histories, velocity counts, missingness flags. **More columns do not automatically make a better model.** So the main question was: *does our feature engineering help us catch more fraud, at a sensible number of false alarms, on later transactions?* **[Jennifer]** broke it into testable hypotheses.

	Question	Who tested it
H1	Do flexible models (trees, networks) rank fraud better than a linear model?	All three
H2	Does the full engineered feature set beat the raw columns?	Jennifer, Jean
H3	Does target encoding (`te_`) help?	Jennifer
H4	Do behaviour/velocity features (`beh_`, `vel_`) help?	Jennifer, Jean
H5	Are the alerts useful at a realistic review budget?	All three
H6	Are these conclusions final?	All three
H7	Does blending two models beat the best single model?	Jennifer, Eric
H8	Is there any sign of leakage?	Jennifer

Two words used throughout. A **baseline** is a reproducible reference model, not a model we assume is weak. An **ablation** removes one group of columns, retrains with everything else fixed, and measures what changes.

2 The data we used

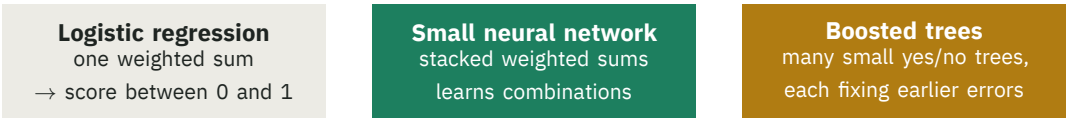
Nobody redid any preprocessing. Every model loads the tables our feature-engineering notebook already saved, because their rules (medians, scalers, encodings) were fitted on the training period only. Redoing them in a new place is an easy way to leak future information.

Split	Train: 472,432 transactions (earliest 80%, fraud rate 3.51%). Validation: the next 59,054 (fraud rate 3.13%). Holdout: the last 59,054 (**not loaded**).
Tree table (537 cols)	Categories and missing values kept as they are. Used by LightGBM, XGBoost and Jean's pipeline.
Numeric table (515 cols)	Every column encoded, gaps filled, standardized. Used by Jennifer's logistic regression, the first XGBoost run and Eric's networks.
Feature sets	`raw` (430 original columns), `all` (537), `all` minus the 3 target-encoding columns (534), `all` minus the 32 behaviour/velocity columns (505).

Sanity checks run first [Jennifer]: features and labels line up row by row, train and validation have the same columns, no row appears in both, and the forbidden columns (`isFraud`, `TransactionID`, `TransactionDT`) are never used as inputs.

Proof that we all used the same data [Eric]: Eric re-ran Jennifer's logistic regression and LightGBM inside his own section and got **exactly** the same numbers (0.4276 and 0.6315). So every comparison in this document is on identical rows.

3 The three kinds of model



- **Logistic regression** adds up weighted columns in a straight line, $p(\text{fraud}) = \sigma(b + w_1 x_1 + \cdots + w_p x_p)$. It cannot learn "small amount *and* new device *and* unusual email" unless someone builds that column by hand. It is the essential *control*: if a fancy model cannot beat it, the complexity is not worth it.
- **Boosted trees** (LightGBM and XGBoost) build many small decision trees in sequence. Each tree asks questions like "Is the amount unusually high?" and later trees focus on the mistakes of earlier ones. They handle missing values and sharp cut-offs natively. LightGBM and XGBoost are two different libraries of the same idea; they differ in how they grow trees and handle categories.
- **Neural networks** put hidden layers between the inputs and the score, so they can learn feature combinations by themselves. Eric's *Small Neural Networks Explained* PDF covers how they work in detail.

4 How we measure

Everyone used the same `evaluate()` function, written by **[Jennifer]** and reused unchanged by Jean and Eric. That is why all our numbers fit in one table.

- **Average precision (AP)**, our main score. It summarizes how precise the alerts are across every catch rate. Random guessing scores the fraud rate, about **0.031** (not 0.5).
- **Recall at 1% FPR**: out of 100 frauds, how many we catch if we may flag at most 1% of legitimate customers.
- **TFPR** (the advisor's ratio): legitimate alerts per fraud caught. "0.5" means one false alert for every two frauds caught.
- **Top-5% capture**: the share of fraud (cases or dollars) found if analysts review only the riskiest 5% of transactions.

One caveat applies to every number below. We use validation both to decide when to stop training and to compare models. That makes validation scores slightly optimistic. The untouched holdout will give the honest number for the final model, scored once.

5 Results at a glance

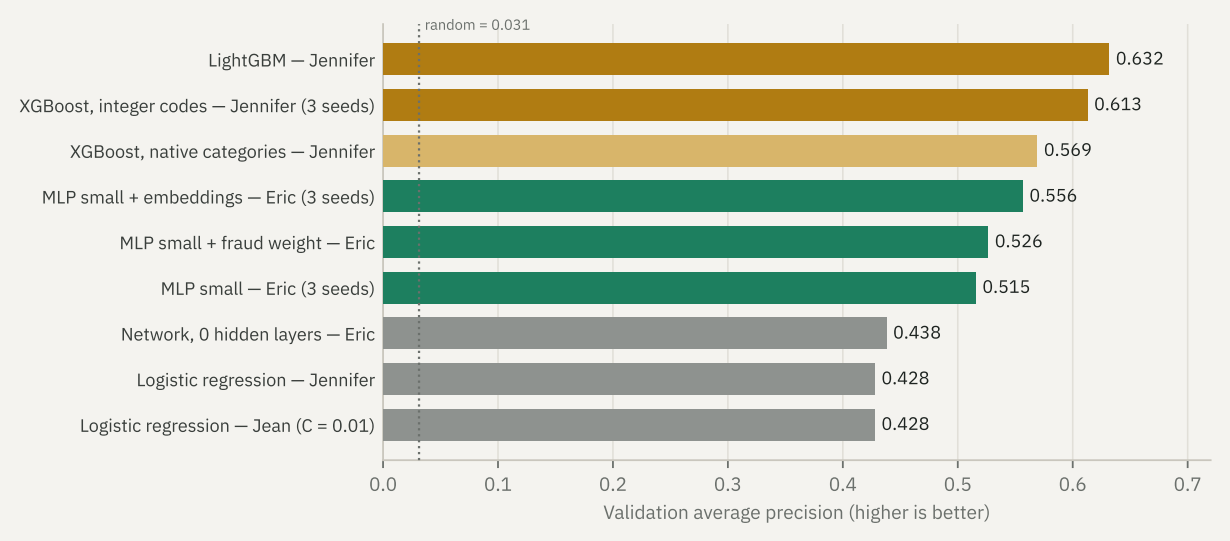


Figure 1: Validation AP for every baseline. Gray: linear models. Emerald: Eric's neural networks. Amber: boosted trees (light amber = XGBoost before its settings were fixed). Models marked "3 seeds" show the average.

Model	AP	ROC-AUC	Recall @1% FPR	TFPR @50%	Top-5%
Logistic regression **[Jean]**	0.428	0.856	36.6%	1.99	51.3%
Logistic regression **[Jennifer]**	0.428	0.867	37.6%	1.98	50.8%
MLP small + embeddings, 3 seeds **[Eric]**	0.556	0.905	50.8%	0.56	61.5%
XGBoost, numeric table **[Jennifer]**	0.610	0.925	53.2%	0.46	67.0%
XGBoost, integer codes, 3 seeds **[Jennifer]**	0.613	0.927	53.8%	0.44	67.4%
LightGBM, `all` **[Jennifer]**	**0.632**	**0.931**	**56.4%**	**0.34**	**68.4%**

Top-5% = share of fraud cases found in the riskiest 5% of transactions.

6 Track A — LightGBM [Jennifer]

LightGBM was trained on the tree table with fixed settings (learning rate 0.05, up to 6,000 trees, early stopping after 100 rounds without improvement). Only the input columns changed between runs, so any difference comes from the features.

6.1 H2–H4: Do our engineered features help? Not consistently.

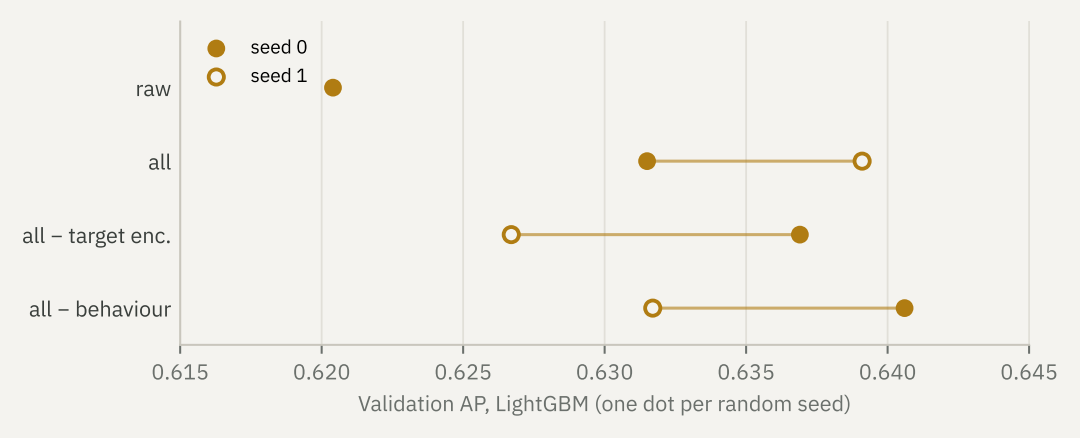


Figure 2: The same LightGBM model, four input sets. On seed 0, removing behaviour features gave the best AP; on seed 1, the full set did. The ordering flips with the random seed.

Feature set	Seed 0	Seed 1	Full minus this, average
`raw` (430 cols)	0.620	—	—
`all` (537 cols)	0.632	0.639	reference
`all` minus target encoding (534)	0.637	0.627	+0.004
`all` minus behaviour/velocity (505)	0.641	0.632	−0.001

- **Engineering as a whole helps a little on seed 0**: `raw` 0.620 vs `all` 0.632. Only one seed of `raw` was run.
- **Removing a family flips direction between seeds.** Removing target encoding: −0.005 on seed 0, +0.012 on seed 1. Removing behaviour: −0.009, then +0.007. The averages are inside the roughly 0.008 AP that LightGBM moves between seeds, so they are **ties**.
- **Important does not mean helpful.** `te_card2` is the 3rd most-used column by gain, yet removing all target encodings did not hurt. Other columns carry the same information.

The address-missing segment. Our Key 2 history was built for payments with no billing address (9% of validation, but 12% of them are fraud). There, keeping behaviour features gives AP 0.806 vs 0.802 without them on seed 0, and 0.812 vs 0.801 on seed 1. That is a small, repeatable benefit in the group it was designed for. Where the address is present, removing behaviour helped on seed 0 (0.550 vs 0.529) but keeping it was slightly better on seed 1 (0.539 vs 0.533).

7 Track B — Logistic regression [Jennifer] [Jean]

We built two linear baselines independently, which turned out to be a useful check on each other.

	Jennifer (Section 5)	Jean (Section B2)
Input	Team numeric table (515 cols)	Team tree table, run through Jean's own preprocessing layer (830 cols)
Settings	`C = 0.1`, fraud rows weighted up (`class_weight = "balanced"`)	`C` tried at 0.01, 0.1, 1; no extra weight on fraud
Chosen by	Fixed in advance	Best validation AP across 9 fits
AP / ROC-AUC	0.4276 / 0.867	0.4276 / 0.856

7.1 Jean's preprocessing layer

Logistic regression needs every input to be a clean number on a similar scale. Jean wrote a layer that learns its rules from the training rows only and then applies them unchanged to validation:

- **Skewed numbers** (counts, amounts, time gaps) are compressed with arcsinh, which acts like a log but keeps zeros and negatives. 332 columns got this.
- **Missing values** are filled with the training median, plus a "was missing" flag for 60 columns where missingness carries information (one flag per distinct missing pattern).
- **Everything is standardized** and then clipped at ± 10 standard deviations, so one extreme later value cannot dominate.
- **Categories become 0/1 columns**. Large ones (email domains, browsers) pool values seen fewer than 200 times into one bucket.
- **Redundant `V` columns are dropped**: 58 of 339 were nearly identical (correlation above 0.95) to one already kept.

7.2 H2/H4 for a linear model: does engineering help? A little.

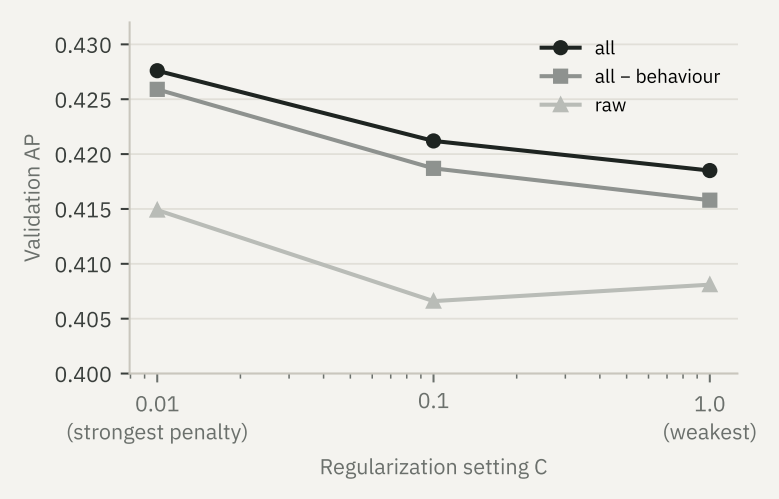


Figure 3: Jean's grid. The strongest penalty (C = 0.01) is best for every feature set, and `all` beats `raw` at every setting.

- **Engineered features add about +0.013 AP** for the linear model (`raw` 0.415 → `all` 0.428). Most of it comes from non-history columns: the behaviour/velocity block adds only +0.002.
- **More regularization was always better**, and the best `C` was the smallest value tried. Trying `C = 0.001` once is the obvious next step.
- **The model's biggest weights** point the same way as the trees: `C1` (+0.97) raises the fraud score; `C14` (−1.29), `C13` (−0.52), debit cards (−0.41) and Product R (−0.38) lower it. A missing `card2` raises it (+0.45). These are associations, not causes.
- **At a fixed 2% review rate**, Jean's model catches 35.5% of fraud with 55.6% of alerts being real fraud. That threshold (0.351) is frozen from validation so it can be checked honestly on the holdout later.

Why the identical AP matters. Two pipelines with different inputs, different preprocessing and different class weighting both land on 0.4276. That strongly suggests **about 0.43 is the ceiling for a straight-line model** on these features. Jennifer's version is slightly better at the strict end (recall at 1% FPR 37.6% vs 36.6%); Jean's gives scores you can read as probabilities, because it does not over-weight fraud (Section 10.4).

8 Track C — XGBoost deep-dive [Jennifer]

The project overview suggested XGBoost, so Jennifer took it from one quick run to a full set of experiments.

8.1 Getting the settings right (the biggest single gain)

The first XGBoost run on the tree table scored only 0.569, far below LightGBM's 0.632. Before concluding "LightGBM is the better library", Jennifer changed one setting at a time.

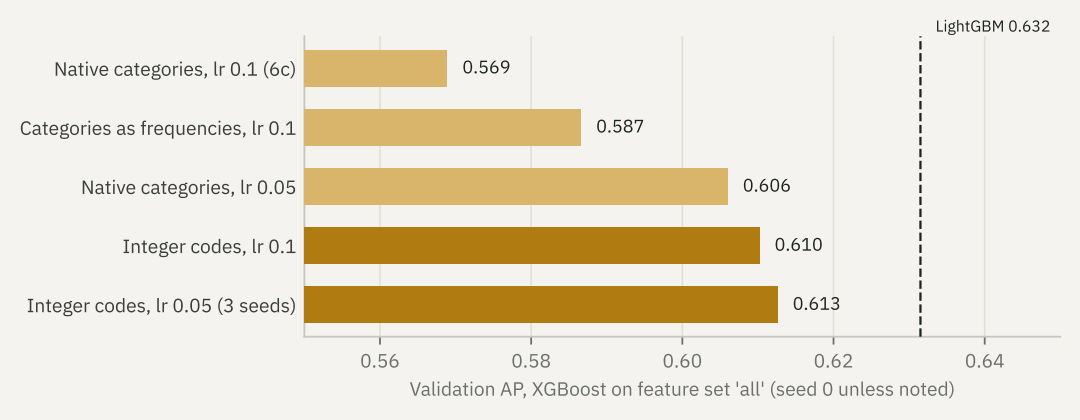


Figure 4: XGBoost on the same 537 columns, changing one setting at a time. Turning categories into integer codes was the largest single fix.

- **Categories were the main problem.** XGBoost's built-in category handling has no smoothing, so on columns with hundreds of values (screen size, browser, email domain) it overfits. Turning categories into integer codes jumped AP from 0.569 to 0.610.
- **A slower learning rate** (0.05 instead of 0.1, with more patience) also helped native categories (0.606), but took 5× longer (507 s vs 105 s). With integer codes it added nothing (0.6102 vs 0.6103).
- **Final setting, 3 seeds**: 0.610, 0.611, 0.616, mean **0.613** ± **0.003**. Very stable.

8.2 H1: Is LightGBM really better? Yes, by about 0.02 AP.

Same-input comparison	XGBoost	LightGBM	Gap
Tree table, seed 0	0.610	0.632	−0.021
Tree table, seed 1	0.611	0.639	−0.028
Numeric table, seed 0 (Follow-up Experiment 2)	0.610	0.619	−0.010

Both tree-table gaps are about three times the seed noise, so LightGBM's lead is real on this validation period. Neither library has been tuned yet, so the honest wording is "LightGBM is about 0.02 AP ahead, untuned".

8.3 Which columns carry the signal?

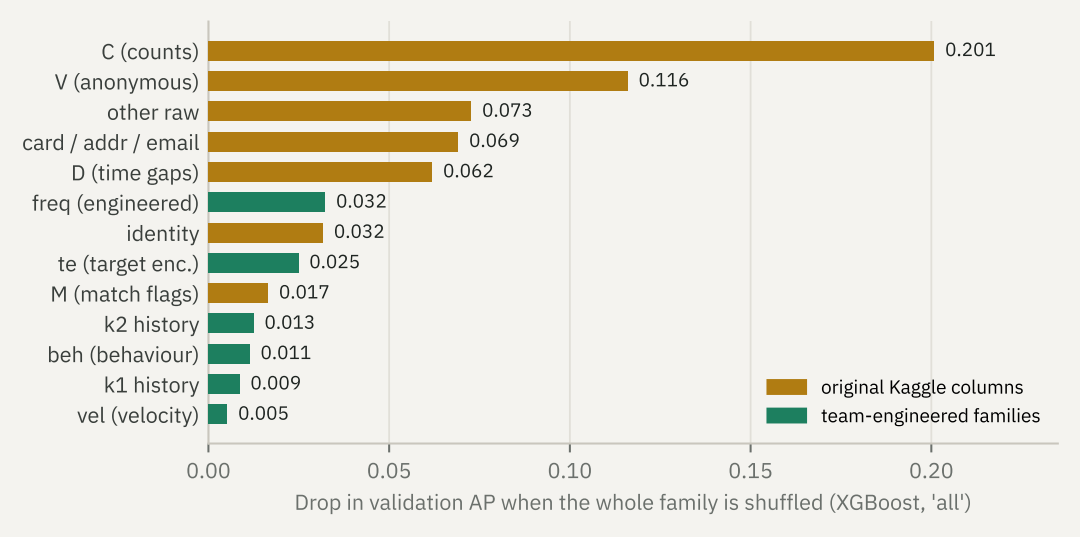


Figure 5: Shuffle a whole family of columns on validation and see how much AP falls. Amber: original Kaggle columns. Emerald: our engineered families.

- **The original `C` count columns and the anonymous `V` block carry most of the signal** (AP drops of 0.20 and 0.12 when shuffled).
- **Our engineered families add a little each**: frequency 0.032, target encoding 0.025, history keys 0.009–0.013, behaviour 0.011, velocity 0.005.
- By SHAP (how much each column moves individual predictions), the top five are `C13`, `te_card2`, `amount_to_product_median`, `C14` and `C1`. Again, `te_card2` is heavily used, yet removing it did not hurt: the information is available elsewhere.

8.4 Is a difference real? The time-block bootstrap

Fraud arrives in bursts, so transactions are not independent. Jennifer's check resamples 100 contiguous blocks of validation time and recomputes the AP difference 300 times. If the 95% interval includes 0, the difference could be noise.

Comparison (XGBoost, native categories)	Difference	95% interval	Verdict
`all` vs `raw`	+0.001	−0.008 to +0.011	Tie
`all` vs `all` minus behaviour	−0.002	−0.010 to +0.005	Tie
`all` vs `all` minus target encoding	−0.012	−0.022 to −0.002	Removing `te_` helped
XGBoost vs LightGBM	−0.063	−0.077 to −0.049	LightGBM better

The target-encoding result is the one to watch: it clears the noise for XGBoost, but not for LightGBM, and it was measured before XGBoost's settings were fixed. It needs a re-run with the final setup over 3 seeds before anyone drops `te_`.

8.5 Does blending help? No.

A *rank blend* turns each model's scores into ranks and averages them.

- **25% XGBoost + 75% LightGBM** ties LightGBM alone (0.6312 vs 0.6315; interval −0.003 to +0.002). Giving XGBoost more weight makes it worse (75%: −0.010, interval below 0).
- **Every blend that included logistic regression was clearly worse** (−0.10 to −0.13 AP). Averaging in a much weaker model drags the result down.

8.6 H8: Any sign of leakage? No.

Jennifer shuffled the training labels and retrained 3 times. With nothing real to learn, the model should score like random guessing, and it did: average AP 0.033 (fraud rate 0.031) and ROC-AUC 0.46 (chance is 0.5). One gap remains: the target-encoding columns were built from the real labels before shuffling, so this test cannot check them. They need their own check in the feature-engineering notebook.

9 Track D — Small neural networks [Eric]

Eric trained networks on the same numeric table as Jennifer's logistic regression, changing one thing at a time. His own PDF has the full explanation; here are the results from the combined run.

Model	Weights	AP	Recall @1% FPR	TFPR @50%
Network, 0 hidden layers (= logistic regression)	516	0.438	38.0%	2.02
MLP tiny (64)	33k	0.497	41.9%	1.21
MLP small (256–128)	166k	0.511	44.9%	1.02
MLP medium (512–256–128)	430k	0.509	44.5%	0.98
MLP small + fraud weight 5.2×	166k	0.526	46.1%	0.84
MLP small + embeddings	266k	**0.551**	**50.1%**	**0.60**

Seed 0. LightGBM on the same scale: AP 0.632, recall 56.4%, TFPR 0.34.

- **Hidden layers help** (yes). The zero-hidden-layer network matches logistic regression (0.438 vs 0.428), so the training code is not secretly better. One hidden layer of 64 neurons already reaches 0.497.
- **A bigger network did not help this time** (no). Medium scored 0.509 vs small 0.511. In Eric's own earlier run medium was ahead, so size is not a reliable lever.
- **Weighting fraud rows helps a little** (promising): 0.511 → 0.526, one seed only.
- **Embeddings give the largest gain** (yes). Columns like `card1` are ID codes stored as numbers; treating them as numbers says card group 9500 is "bigger" than 7919, which is meaningless. An embedding gives each common value its own small learned vector instead.

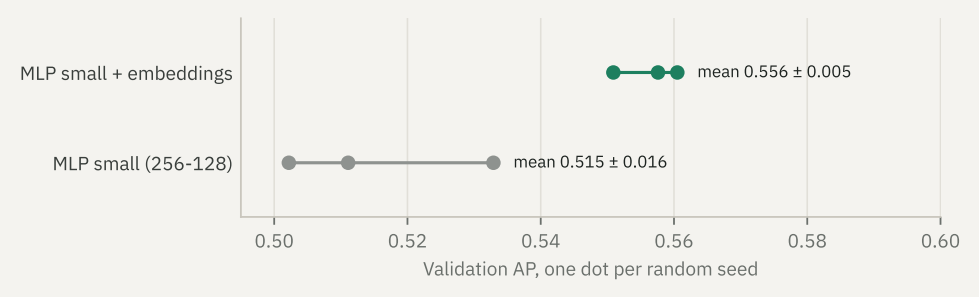


Figure 6: Three random seeds each. The worst embedding run (0.551) still beats the best plain run (0.533). The average gain is 0.041.

Blending the network into LightGBM made things worse (10% network: −0.004, interval −0.008 to +0.0003; 20% and above clearly below 0). The reason is in Section 10.3: the network mostly catches frauds LightGBM already catches.

10 Putting the tracks together

10.1 H5: Are the alerts useful? Only LightGBM stays under 10 : 1 at 90%.

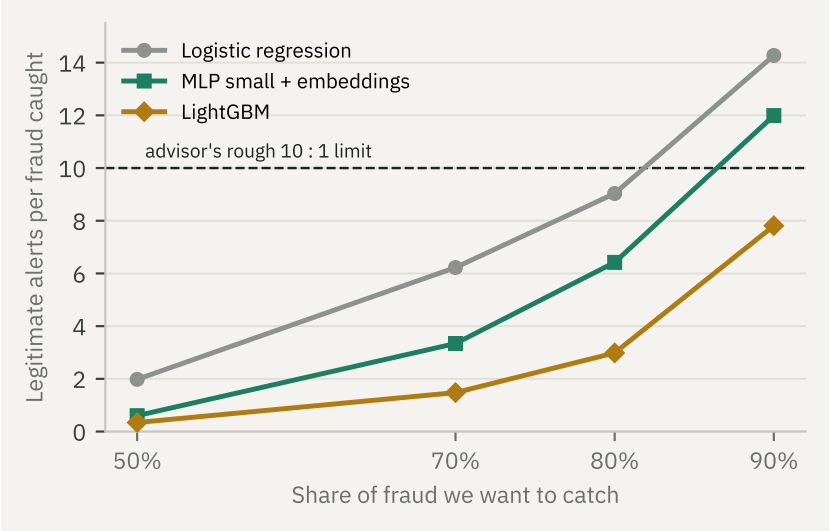


Figure 7: Legitimate alerts created per fraud caught, as we try to catch more fraud. Lower is better. From Eric's comparison (Section D13a).

To catch this share of fraud	Logistic regr.	Best network	LightGBM
50%	1.98	0.60	**0.34**
70%	6.23	3.34	**1.48**
80%	9.03	6.42	**2.98**
90%	14.27	11.99	**7.81**
Share of all transactions reviewed at 90%	43%	37%	**25%**

The advisor's rough target is about 10 legitimate alerts per fraud. LightGBM stays under it all the way to 90% detection; XGBoost (native categories) reaches 9.9 there, right at the line. The exact detection target and review capacity still need to be confirmed with the advisor.

10.2 Where every model is weak

Read across a row, not down a column: a segment with more fraud gets a higher AP even from a random model.

Segment	Rows	Fraud rate	Logistic	Network	LightGBM
Product C	5,628	11.8%	0.732	0.765	0.804
Product H	1,561	5.5%	0.465	0.516	0.625
Product R	2,558	4.7%	0.724	0.805	0.858
Product S	1,064	6.6%	0.323	0.579	0.668
Product W	48,243	1.9%	0.116	0.351	0.461
Identity present	10,360	8.9%	0.684	0.734	0.787
Identity missing	48,694	1.9%	0.117	0.352	0.462
Address missing	5,486	12.1%	0.738	0.767	0.806

Product W and transactions without identity data are the weak spot for every model, and each covers about 82% of validation. Jennifer's error analysis **[Jennifer]** points the same way: reviewing the riskiest 5% (2,953 transactions), XGBoost catches 1,185 frauds and misses 666 (36%), worth about $108k of $268k fraud dollars. Over half of the misses are Product W (53%) or have no identity data (53%). The largest

dollar misses are $1.1k–$3.1k Product W transactions that received very low scores.

10.3 Do the models catch the same fraud?

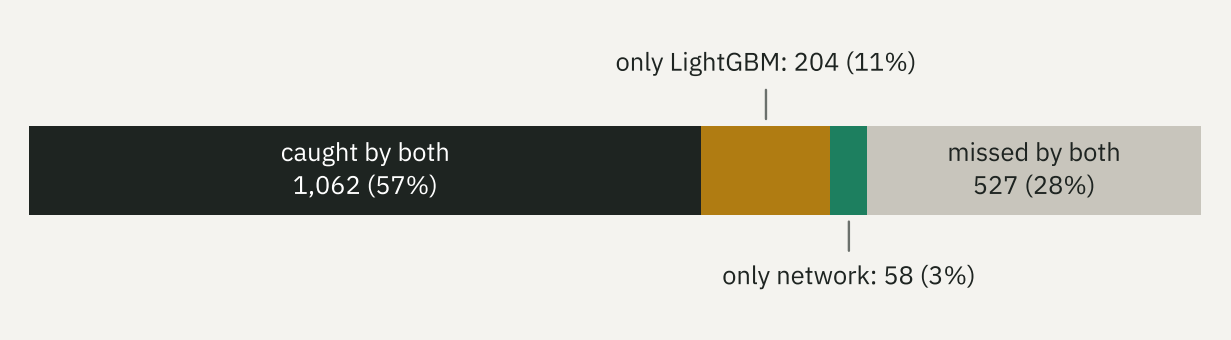


Figure 8: The 1,851 validation frauds, sorted by which model catches them in its riskiest 5% (Eric, Section D17).

The network adds only 58 frauds that LightGBM misses, and the two models' rankings are strongly correlated (0.72). **527 frauds (28%) are missed by both.** That is why blending does not help, and why the next improvement probably needs new information about these transactions rather than another algorithm.

10.4 Can the scores be read as probabilities?

Calibration asks: if a model says 0.30, are about 30% of those transactions really fraud? The real fraud rate is 3.1%.

- **Unweighted models are close**: Eric's embedding network averages 0.032, LightGBM 0.024, and Jennifer's XGBoost has a Brier score of 0.019 (no-skill 0.030). Jean's logistic regression is also unweighted by design.
- **Weighting fraud rows inflates scores**: Jennifer's balanced logistic regression averages 0.287, about 9× too high; Eric's weighted network averages 0.085.
- Ranking is unaffected, so for a review budget this is fine. To show analysts a "probability of fraud", use an unweighted model or recalibrate.

11 Decision log

Question	What we can say now	Next action
H1 Flexible models?	Yes. Trees ≫ networks ≫ linear (0.632 / 0.556 / 0.428). LightGBM leads XGBoost by about 0.02.	Tune XGBoost (Optuna, 6g) and re-compare.
H2 Full engineered set?	Small gain for both LightGBM (+0.011, one seed) and Jean's linear model (+0.013). XGBoost: tie.	Re-test with final XGBoost over 3 seeds.
H3 Target encoding?	Inconsistent: helps or hurts depending on seed and model. Heavily used but not clearly useful.	Same; check out-of-time encoding with the advisor.
H4 Behaviour / velocity?	No consistent overall effect. Small repeatable help where the address is missing.	Same; keep for now.
H5 Useful alerts?	Only LightGBM stays under 10 : 1 at 90% detection.	Confirm the target with the advisor.
H6 Final?	No. Development findings on validation only.	Freeze model and features; score the holdout once.
H7 Blending?	No. Every blend tied or lost.	Use one model.
H8 Leakage?	None found in label-free features. `te_` not covered.	Rebuild `te_` from shuffled labels.

12 Things we learned that will save you time

- **PyTorch and LightGBM in one process can freeze on macOS. [Eric]** Each ships its own threading library. In the combined notebook, restart the kernel before Eric's section; it reloads everything it needs.
- **"No manifest.json found" means a path problem, not missing data.** The setup cell looks for `outputs/...` relative to the folder Jupyter runs from. Keep the notebook in `notebooks/` next to the `outputs` folder.
- **Check the defaults before blaming the algorithm. [Jennifer]** XGBoost's "gap" to LightGBM was mostly how it handled categories. One change took it from 0.569 to 0.610.
- **Never decide from one seed.** The "best" LightGBM feature set changed when only the random seed changed.
- **High importance ≠ useful.** `te_card2` ranks near the top by gain and SHAP, yet removing it did not hurt. Use ablations to decide what to keep.
- **Save predictions and make long cells resumable.** The XGBoost seeds and LightGBM ablations save each fit to disk, so a crash or restart loses nothing and later analyses need no retraining.
- **Keep a "dumb" control in every track.** Logistic regression for the trees, the zero-hidden-layer network for Eric's models. It separates "the model is better" from "my code is different".

13 Limitations and next steps

Limitations. Validation is used for early stopping and model choice, so it is optimistic. Most comparisons use one or two seeds. Nothing is tuned yet. There is only one validation period of about three weeks, and fraud patterns drift. Several planned XGBoost experiments were not run: incremental families (6e), class

weighting (6f), Optuna tuning (6g), the 33 new label-free features (6k) and the leave-one-family-out table (6p).

Proposed next steps.

1. Tune XGBoost and LightGBM equally, then re-compare over 3 seeds.
2. Run the leave-one-family-out ablation (6p) over 3 seeds. It is the table the advisor will want, and it settles H2–H4.
3. Test the 33 label-free features Jennifer built (6k), aimed at Product W and no-identity transactions.
4. Study the 527 frauds every model misses and plan features for them as a team.
5. Rebuild the target encodings from shuffled labels to finish the leakage check.
6. When the team freezes the final model, score the holdout **once**. Jean's holdout step (B2.5) is ready and switched off until then.

How to reproduce

1. Run `BTT_Mastercard_Feature_Engineering_Final.ipynb` first; it writes `outputs/feature_engineering/fe_final_v1/`.
2. Put `BTT_Mastercard_Baseline_Models_Combined.ipynb` in `notebooks/` and run Sections 0–14 and B2 top to bottom. Long steps save to disk and are skipped on re-runs.
3. On macOS, restart the kernel, then run from Section D1 to the end (Eric's part and the all-team scoreboard).
4. Results are saved under `fe_final_v1/baselines/student_explained_v2/` (Jennifer, Jean, and the all-team scoreboard) and `fe_final_v1/neural_nets/eric_v1/` (Eric).